

Database Design Document

CS6650 Assignment 3 — Fan Zhang

1. Database Choice Justification

PostgreSQL 15 was selected because it natively supports all four core query patterns: GROUP BY, COUNT DISTINCT, BETWEEN range queries, and ORDER BY on non-primary-key columns. Alternatives were rejected for the following reasons:

Alternative	Reason Rejected
DynamoDB	No GROUP BY or COUNT DISTINCT; Core Q3 (active user count) impossible without full scan + client-side aggregation
Cassandra	Top-N analytics require full partition scans; ORDER BY restricted to clustering keys only
MongoDB	Aggregation pipeline adds complexity with no performance advantage for this append-only workload

Additionally, PostgreSQL's synchronous_commit = off configuration and JdbcTemplate.batchUpdate() with ON CONFLICT DO NOTHING enabled sustained write throughput of 1,200–1,250 msg/s on a t3.small instance — meeting the assignment's high-throughput persistence requirement without requiring a larger instance type.

2. Complete Schema Design

Column	Type	Notes
id	BIGSERIAL PK	Surrogate key; avoids UUID fragmentation in B-tree indexes
message_id	VARCHAR(36) UNIQUE	Client UUID; enables ON CONFLICT DO NOTHING idempotent writes
room_id	VARCHAR(20) NOT NULL	Cardinality 1-20; composite index leading column
user_id	VARCHAR(10) NOT NULL	Cardinality 1-100,000; high selectivity for user queries
username	VARCHAR(20) NOT NULL	Denormalized for query performance
message	TEXT NOT NULL	Unbounded body; TEXT avoids VARCHAR length overhead
event_time	TIMESTAMPTZ NOT NULL	Client creation time; enables BETWEEN range queries and DATE_TRUNC
message_type	VARCHAR(10) NOT NULL	TEXT/JOIN/LEAVE; too low selectivity for index
server_id / client_ip	VARCHAR (nullable)	Routing and audit metadata
published_at	TIMESTAMPTZ	RabbitMQ publish timestamp for pipeline latency tracking
received_at	TIMESTAMPTZ DEFAULT NOW()	Consumer pipeline entry time; latency ≈ received_at – event_time

The schema follows an append-only write pattern: no UPDATE or DELETE operations occur during normal operation, which eliminates lock contention and dead tuple accumulation (n_dead_tup = 0 confirmed across all three load tests). The timestamp field is named event_time (TIMESTAMPTZ) rather than the PostgreSQL reserved word timestamp, avoiding type name conflicts while enabling timezone-aware BETWEEN range queries and DATE_TRUNC aggregation for the Analytics API.

3. Indexing Strategy

Index	Columns	Supports Queries	Write Impact
messages_pkey	id	Internal lookups	Minimal
messages_message_id_key	message_id (UNIQUE)	ON CONFLICT duplicate detection	Checked every INSERT
idx_room_event_time	(room_id, event_time)	Core Q1: room messages in time range	Medium — B-tree maintained per INSERT
idx_user_event_time	(user_id, event_time)	Core Q2: user history; Core Q4: rooms for user	Medium — B-tree maintained per INSERT

Index	Columns	Supports Queries	Write Impact
idx_event_time	event_time	Core Q3: active users; Analytics Q1: msg/sec stats	Light

Composite indexes (room_id, event_time) and (user_id, event_time) cover both equality and range predicates in a single B-tree traversal, eliminating a secondary sort step. message_type was excluded: with only 3 possible values, an index scan + heap fetch is slower than a sequential scan.

explain-analyze.txt results at 500K rows — all Core Query targets met:

Query	Execution Plan	Time	Target
Core Q1 (room + time range)	Bitmap Index Scan on idx_room_event_time	2.964ms	< 100ms ✓
Core Q2 (user history)	Bitmap Index Scan on idx_user_event_time	0.054ms	< 200ms ✓
Core Q3 (COUNT DISTINCT)	Seq Scan (optimizer; all rows in window)	35.589ms	< 500ms ✓
Core Q4 (rooms for user)	Bitmap Index Scan on idx_user_event_time	0.087ms	< 50ms ✓

Core Q3 uses a sequential scan because the query window covers nearly all rows, making the seq scan cheaper than index traversal. At production scale with selective time windows, the optimizer switches to idx_event_time.

4. Scaling Considerations

Scale	Bottleneck	Solution
10M rows	Index B-tree depth; buffer cache pressure	Vertical scale to t3.medium (4GB RAM); increase shared_buffers to 1GB
100M rows	Single-instance write ceiling; index maintenance cost	Partition table by event_time (monthly); partition pruning reduces index scan range
Multi-region writes	PostgreSQL single-primary limitation	Citus sharding by room_id; distributes writes while preserving SQL compatibility
Read-heavy analytics	Analytical queries compete with write I/O	Read replicas for Metrics API; offloads GROUP BY aggregations from write primary

The schema is partition-ready: event_time as TIMESTAMPTZ enables range partitioning; room_id's bounded cardinality (1-20) is suitable for hash distribution with Citus.

Error handling is implemented at three layers: (1) idempotent writes via ON CONFLICT (message_id) DO NOTHING handle duplicate messages from RabbitMQ nack-requeue; (2) DbCircuitBreaker (CLOSED/OPEN/HALF_OPEN) stops writes when PostgreSQL is unavailable, routing failed batches to chat.dlq for manual replay; (3) exponential backoff retry (1s→2s→4s, max 3 attempts) handles transient DB failures before tripping the circuit breaker.

5. Backup and Recovery

Logical backups use pg_dump with compressed custom format (-F c), reducing backup size by ~65% vs plain SQL. Daily backups are retained for 7 days:

```
pg_dump -U chatflow -d chatflow -F c -f /backup/chatflow_$(date +%Y%m%d).dump
find /backup -name 'chatflow_*.dump' -mtime +7 -delete
```

Recovery procedure on instance failure:

- Launch new EC2 instance; run setup.sh to install PostgreSQL 15 and recreate schema
- Restore: pg_restore -U chatflow -d chatflow -F c /backup/chatflow_YYYYMMDD.dump
- Update JDBC URL in consumer-v3 and server-v2 application.yml; restart services

Estimated RTO: under 30 minutes for current data volume (186MB table + 103MB indexes). For production, AWS RDS Multi-AZ would reduce RTO to under 2 minutes via automatic failover and point-in-time recovery.